

# *Algorithm Analysis And Data Structures*

Prof. Dr. Ayla ŞAYLI

[http://avesis.yildiz.edu.tr/sayli/  
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)

## 3.2.2. Linked Lists

- The linked list consists of a series of structures, which are not necessarily adjacent in memory.
- Each structure contains the element and a pointer to a structure containing its successor. We call this the *next* pointer.

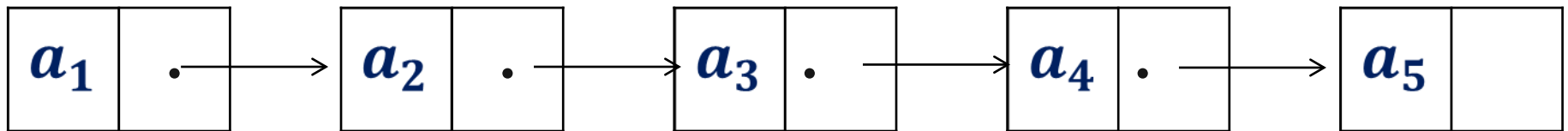


Figure 3.1 A linked list

## 3.2.2. Linked Lists

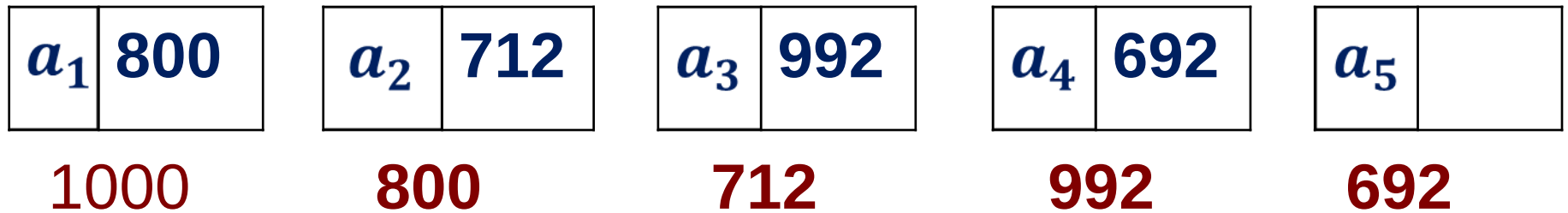


Figure 3.2 Linked list with actual pointer values

## 3.2.2. Linked Lists - Deletion

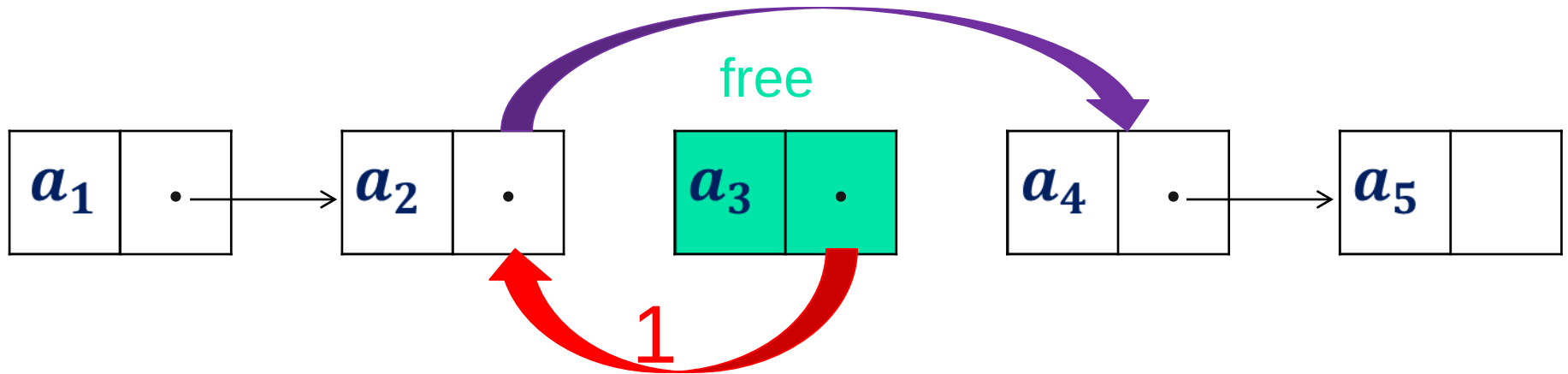
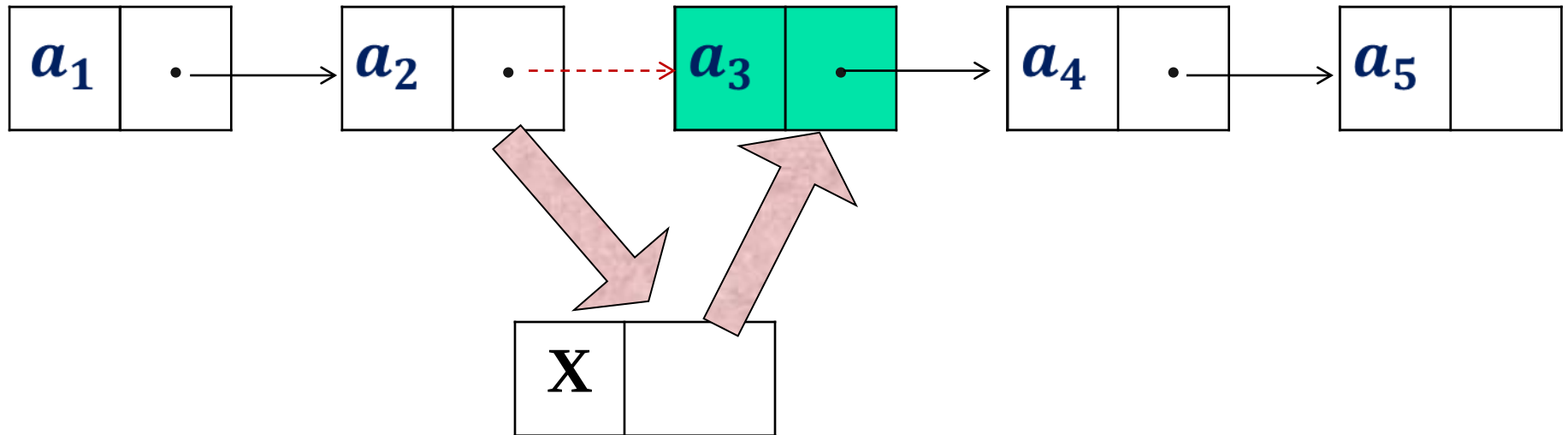


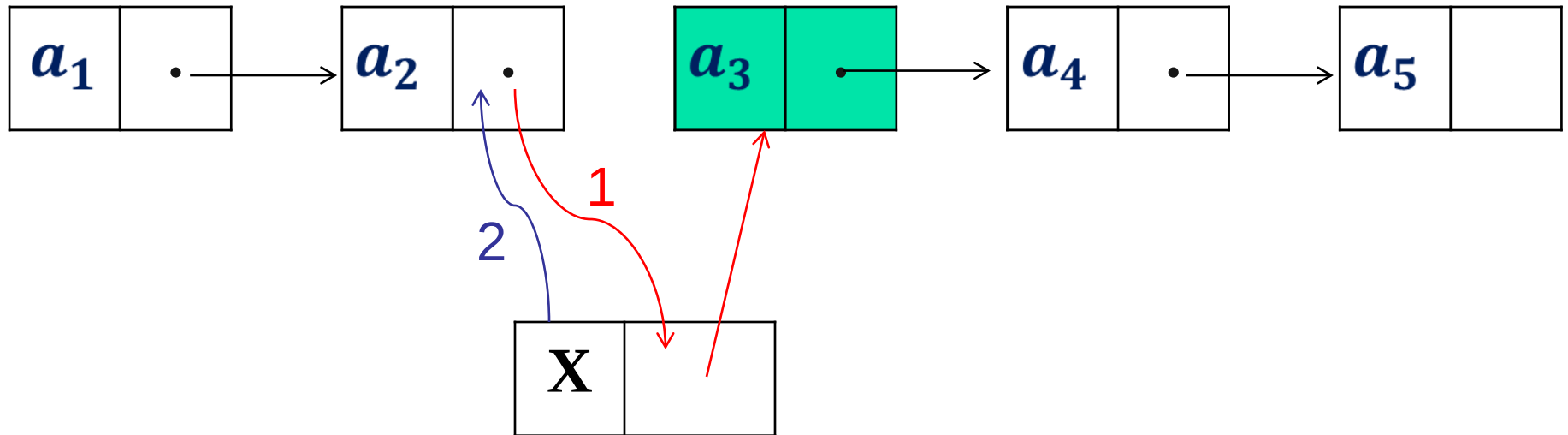
Figure 3.3 Deletion from a linked list

## 3.2.2. Linked Lists -Insert



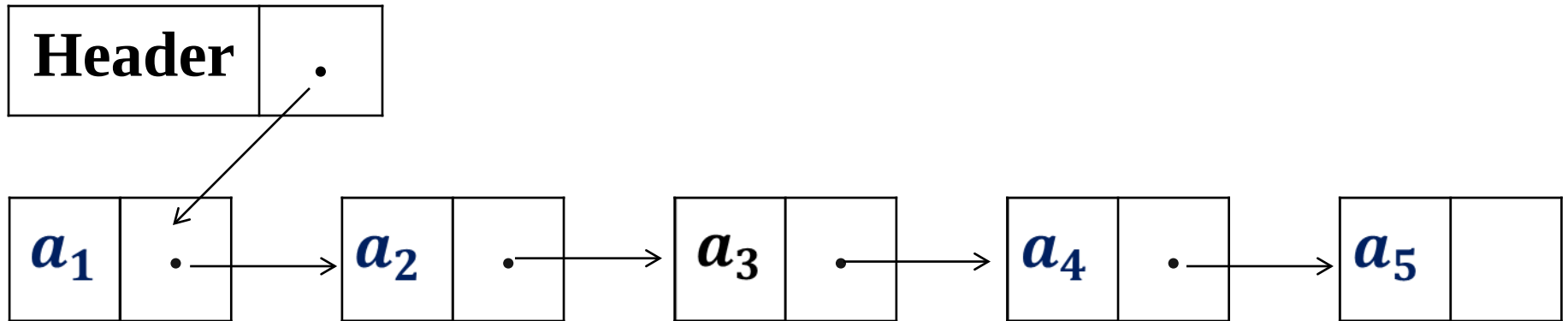
**Figure 3.4 Insertion into a linked list**

## 3.2.2. Linked Lists -Insert



**Figure 3.4 Insertion into a linked list**

## 3.2.3. Programming Details



**Figure 3.5 Linked list with a header**

## 3.2.3. Programming Details

```
typedef struct node *node_ptr;  
struct node  
{  
    element_type element;  
    node_ptr next;  
};  
typedef node_ptr LIST;  
typedef node_ptr position;
```

**Figure 3.6 Type declarations for linked lists**



## 3.2.3. Programming Details



Figure 3.7 Empty list with header

```
int is_empty ( LIST L )  
{  
    return( L->next == NULL );  
}
```

Figure 3.8 Function to test whether a linked list is empty

## 3.2.3. Programming Details

```
int is_last( position p, LIST L )
{
    return( p->next == NULL );
}
```

**Figure 3.9 Function to test whether current position is the last in a linked list**

/\* Return position of x in L; NULL if not found \*/

```
position find ( element_type x, LIST L )
{
    position p;

    p = L->next;
    while( (p != NULL) && (p->element != x) )
        p = p->next;
    return p;
}
```

**Figure 3.10 Find routine**

## 3.2.3. Programming Details

```
void delete( element_type x, LIST L )
{
    position p, tmp_cell;

    p = find_previous( x, L );
    if( p->next != NULL )
    {
        tmp_cell = p->next;
        p->next = tmp_cell->next;
        free( tmp_cell );
    }
}
```

**Figure 3.11 Deletion routine for linked lists**

## 3.2.3. Programming Details

```
position find_previous( element_type x, LIST L )
{
    position p;

    p = L;
    while( (p->next != NULL) && (p->next->element != x) )
        p = p->next;
    return p;
}
```

**Figure 3.12 Find\_previous--the find routine for use with delete**

## 3.2.3. Programming Details

```
void insert( element_type x, LIST L, position p )
{
    position tmp_cell;

    tmp_cell = (position) malloc( sizeof (struct node) );
    if( tmp_cell == NULL )
        fatal_error("Out of space!!!");
    else
    {
        tmp_cell->element = x;
        tmp_cell->next = p->next;
        p->next = tmp_cell;
    }
}
```

**Figure 3.13 Insertion routine for linked lists**

## 3.2.3. Programming Details

```
void delete_list( LIST L )
{
    position p, tmp;

    p = L->next; /* header assumed */
    L->next = NULL;
    while( p != NULL )
    {
        tmp = p->next;
        free( p );
        p = tmp;
    }
}
```

**Figure 3.15 Correct way to delete a list**

## 3.2.5. Doubly Linked Lists

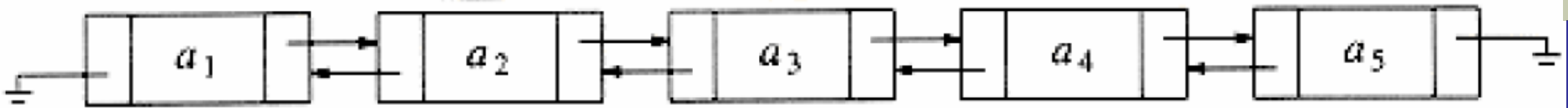


Figure 3.16 A doubly linked list

## 3.2.6. Circularly Linked Lists

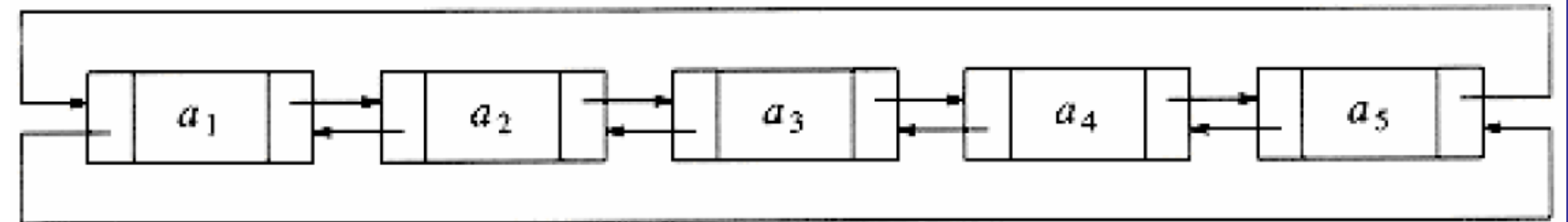


Figure 3.17 A double circularly linked list

## 3.2.7. Examples

*Example :* Two single-variable polynomials are given as

$$p_1(x) = 10x^{1000} + 5x^{14} + 1$$

$$p_2(x) = 3x^{1990} - 2x^{1492} + 11x + 5$$

a) Find  $p_1(x) + p_2(x)$

b) Find  $p_1(x) * p_2(x)$



# By the Use of Array

typedef struct

```
{    int coeff_array[ MAX_DEGREE+1 ];  
    unsigned int high_power;  
} *POLYNOMIAL;
```

**Figure 3.18 Type declarations for array implementation of the polynomial ADT**

void zero\_polynomial( POLYNOMIAL poly )

```
{    unsigned int i;  
  
    for( i=0; i<=MAX_DEGREE; i++ )  
        poly->coeff_array[i] = 0;  
    poly->high_power = 0;  
}
```

**Figure 3.19 Procedure to initialize a polynomial to zero**

# By the Use of Array

```
a) void add_polynomial ( POLYNOMIAL poly1,  
    POLYNOMIAL poly2, POLYNOMIAL poly_sum )  
{   int i;  
    zero_polynomial( poly_sum );  
    poly_sum->high_power = max( poly1->high_power,  
                                poly2->high_power);  
    for( i=poly_sum->high_power; i>=0; i-- )  
        poly_sum->coeff_array[i] = poly1->coeff_array[i]  
                                + poly2->coeff_array[i];  
}
```

**Figure 3.20 Procedure to add two polynomials**

# By the Use of Array

```
b) void mult_polynomial ( POLYNOMIAL poly1,  
                          POLYNOMIAL poly2, POLYNOMIAL poly_prod )  
{ unsigned int i, j;  
  
  zero_polynomial( poly_prod );  
  poly_prod->high_power = poly1->high_power + poly2->high_power;  
  if( poly_prod->high_power > MAX_DEGREE )  
    error("Exceeded array size");  
  else  
    for( i=0; i<=poly->high_power; i++ )  
      for( j=0; j<=poly2->high_power; j++ )  
        poly_prod->coeff_array[i+j] +=  
          poly1->coeff_array[i] * poly2->coeff_array[j];  
}
```

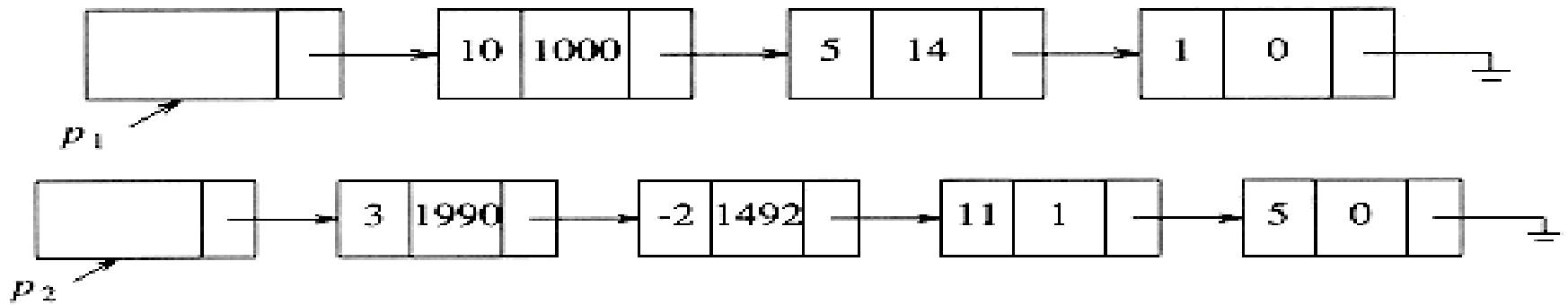
**Figure 3.21 Procedure to multiply two polynomials**

# By the Use of Array

```
b) void mult_polynomial ( POLYNOMIAL poly1,  
                          POLYNOMIAL poly2, POLYNOMIAL poly_prod )  
{ unsigned int i, j;  
  
  zero_polynomial( poly_prod );  
  poly_prod->high_power = poly1->high_power + poly2->high_power;  
  if( poly_prod->high_power > MAX_DEGREE )  
    error("Exceeded array size");  
  else  
    for( i=0; i<=poly->high_power; i++ )  
      for( j=0; j<=poly2->high_power; j++ )  
        poly_prod->coeff_array[i+j] +=  
          poly1->coeff_array[i] * poly2->coeff_array[j];  
}
```

**Figure 3.21 Procedure to multiply two polynomials**

# By the Use of List ADT



```
typedef struct node *node_ptr;
```

```
struct node
```

```
{    int coefficient;
```

```
    int exponent;
```

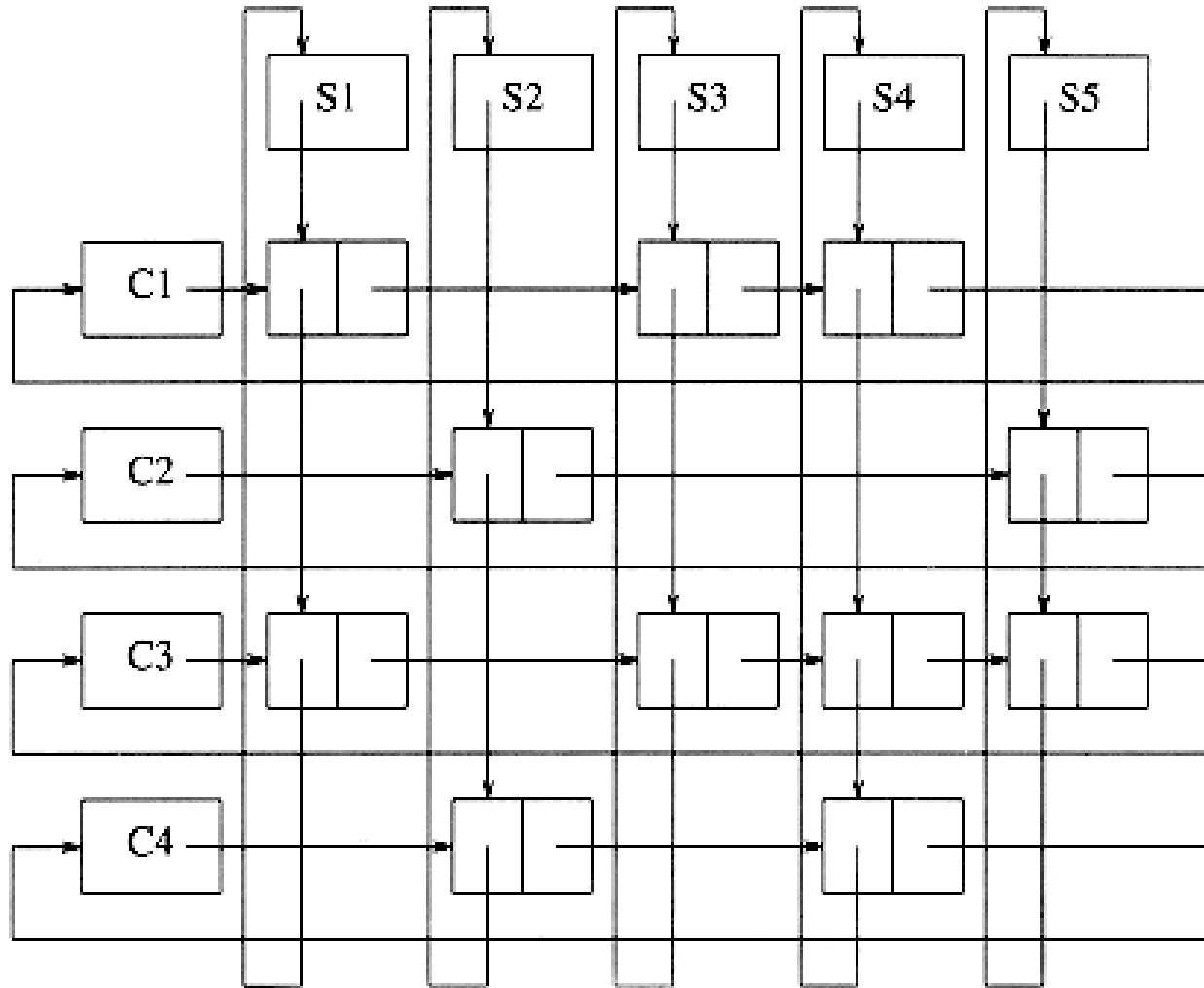
```
    node_ptr next;
```

```
};
```

```
typedef node_ptr POLYNOMIAL; /* keep nodes sorted by exponent */
```

**Figure 3.23 Type declaration for linked list implementation of the Polynomial ADT**

# Multilists



**Figure 3.27 Multilist implementation for registration problem**

# 3.3. The Stack ADT

# 3.4. The Queue ADT



# *Algorithm Analysis And Data Structures*

Questions and Answers

Thank you ...

**Prof. Dr. Ayla ŞAYLI**

[http://avesis.yildiz.edu.tr/sayli/  
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)